

1. C语言简介

C语言是一种通用的编程语言，广泛用于系统软件和应用软件的开发。

C语言特点：

- 接近底层硬件，执行效率高
- 语法简洁，运算符丰富
- 可移植性强
- 是嵌入式开发的首选语言

学习目标：

- 操作硬件
- 驱动开发（基于内核基础）

2. 第一个C程序

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

程序结构说明：

- `#include <stdio.h>` - 包含标准输入输出头文件
- `int main(void)` - 主函数，程序入口
- `return 0` - 程序正常结束

3. 数据类型

基本数据类型

类型	字节	格式符	取值范围
char	1	%c	-128 ~ 127
short	2	%hd	-32768 ~ 32767
int	4	%d , %i	-2147483648 ~ 2147483647
long	4	%ld , %u , %lld	同int (32位系统)
float	4	%f , %g	单精度浮点数
double	8	%lf , %llf , %e	双精度浮点数
long double	12	%Lf	扩展精度浮点数

char类型详解

// char 占1个字节 (8位)
// 最高位是符号位：0表示正数，1表示负数
// 取值范围：-128 ~ 127

```
char c = 'A';           // 字符  
char c = 65;            // ASCII码值
```

变量定义

格式：

数据类型 变量名 = 值；

示例：

```
int age = 25;  
float score = 95.5;  
char grade = 'A';
```

sizeof运算符

sizeof() 用于获取数据类型或变量占用的字节数：

```
#include <stdio.h>

int main(void)
{
    printf("char: %lu byte\n", sizeof(char));
    printf("short: %lu bytes\n", sizeof(short));
    printf("int: %lu bytes\n", sizeof(int));
    printf("long: %lu bytes\n", sizeof(long));
    printf("float: %lu bytes\n", sizeof(float));
    printf("double: %lu bytes\n", sizeof(double));

    int a = 100;
    printf("a: %lu bytes\n", sizeof(a));

    return 0;
}
```

格式化输出（printf）

格式符	说明
%c	字符
%d , %i	十进制整数
%ld , %lld	长整型
%u	无符号整数
%f	浮点数
%lf	双精度浮点数
%e	科学计数法
%o	八进制
%x , %X	十六进制
%#o	带前缀的八进制

格式符	说明
%#x , %#X	带前缀的十六进制
%p	地址
%s	字符串

4. 运算符

算术运算符

运算符	说明	示例
+	加	a + b
-	减	a - b
*	乘	a * b
/	除	a / b
%	取模（取余）	a % b
++	自增	a++ 或 ++a
--	自减	a-- 或 --a

自增自减详解：

```
int a = 5;

// 后缀自加（先赋值，再自加）
int b = a++;    // b = 5, a = 6

// 前缀自加（先自加，再赋值）
int c = ++a;    // c = 7, a = 7
```

关系运算符

运算符	说明	结果
>	大于	成立：1，不成立：0
>=	大于等于	成立：1，不成立：0
<	小于	成立：1，不成立：0
<=	小于等于	成立：1，不成立：0
==	等于	成立：1，不成立：0
!=	不等于	成立：1，不成立：0

逻辑运算符

运算符	说明	规则
&&	逻辑与	所有条件都为真（非0），结果为真
	逻辑或	只要有一个条件为真（非0），结果为真
!	逻辑非	真变假，假变真

```
// 逻辑与
int result = (a > 0) && (b > 0); // a和b都大于0时为真

// 逻辑或
int result = (a > 0) || (b > 0); // a或b有一个大于0即为真

// 逻辑非
int result = !(a > 0); // 如果a>0为真，则结果为假
```

条件运算符（三目运算符）

```
// 格式：条件 ? 真值 : 假值

int max = (a > b) ? a : b; // 取a和b中的较大值
```

练习：判断闰年

```
// 能被4整除但不能被100整除，或者能被400整除
int isLeapYear = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
```

位运算符

位运算符直接操作二进制位：

运算符	说明	规则
&	位与	都为1才为1： 1&1=1 ， 1&0=0 ， 0&1=0 ， 0&0=0
	位或	有1就为1： 1 1=1 ， 1 0=1 ， 0 1=1 ， 0 0=0
~	取反	0变1， 1变0
^	异或	相同为0，不同为1： 1^0=1 ， 0^1=1 ， 1^1=0 ， 0^0=0
<<	左移	高位丢掉，低位补0
>>	右移	低位丢掉，高位补符号位（算术右移）或补0（逻辑右移）

位运算示例：

```
int a = 5;           // 二进制：0101
int b = 3;           // 二进制：0011

a & b = 1;           // 0101 & 0011 = 0001
a | b = 7;           // 0101 | 0011 = 0111
a ^ b = 6;           // 0101 ^ 0011 = 0110
~a = -6;             // 取反

a << 1 = 10;         // 左移1位，相当于乘以2
a >> 1 = 2;          // 右移1位，相当于除以2
```

原码、反码、补码

计算机内部使用补码存储数据：

正数：

- 原码 = 补码 = 反码
- 例如：5 的原码 = 0000...0101

负数：

- 取绝对值 → 正数 → 原码 → 取反 → +1 → 补码
- 例如：-1 的补码计算过程：

$| -1 | = 1 \rightarrow 0000 \dots 0001 \rightarrow \text{取反: } 1111 \dots 1110 \rightarrow +1: 1111 \dots 1111$

符号位：

- 最高位：0表示正数，1表示负数

移位类型：

- **算术移位**：考虑符号位，不会把正数移成负数
- **逻辑移位**：不考虑符号位，可能会把正数移成负数

赋值运算符

运算符	说明
=	赋值
+=	加后赋值
-=	减后赋值
*=	乘后赋值
/=	除后赋值
%=	取模后赋值
&=	位与后赋值
=	位或后赋值
^=	异或后赋值
<<=	左移后赋值
>>=	右移后赋值

```
// 示例
int a = 10;
a += 5;    // 等价于 a = a + 5, 结果 a = 15
a -= 3;    // 等价于 a = a - 3, 结果 a = 12
a <= 1;    // 等价于 a = a < 1, 结果 a = 24
```

5. 输入输出

printf - 格式化输出

```
#include <stdio.h>

int main(void)
{
    int age = 25;
    float score = 95.5;
    char grade = 'A';

    printf("年龄: %d\n", age);
    printf("成绩: %.2f\n", score);
    printf("等级: %c\n", grade);

    return 0;
}
```


scanf - 格式化输入

```
#include <stdio.h>

int main(void)
{
    int age;
    float score;

    printf("请输入年龄: ");
    scanf("%d", &age);    // 注意: 需要使用取地址符 &

    printf("请输入成绩: ");
    scanf("%f", &score);

    printf("年龄: %d, 成绩: %.2f\n", age, score);

    return 0;
}
```

注意： 使用scanf时，变量前需要加 & 取地址符。

6. 数据类型转换

显式转换（强制类型转换）

```
// 格式: (目标类型) 变量名

int a = 10;
float b = (float)a;    // 将int强制转换为float
double c = (double)a; // 将int强制转换为double
```

隐式转换（自动类型转换）

转换方向：

char → short → int → long → double ← float

隐式转换发生场景：

```
// 1. 运算转换
int a = 5;
double b = 2.5;
double result = a + b;    // a 自动转换为 double

// 2. 赋值转换
int a = 10;
float b = 3.14;
a = b;    // b 自动转换为 int, a = 3

// 3. 函数传参
void func(int x);
float f = 3.14;
func(f);    // f 自动转换为 int

// 4. 函数返回
float func() {
    return 3.14;
}
int a = func();    // 返回值自动转换为 int
```

7. C语言注释

注释类型

```
// 单行注释

/*
 * 多行注释
 * 可以跨越多行
 */
```

注释规则：

1. 注释不可以嵌套
2. 注释可以放在行的上面或行的后面，不能放在语句中间

使用Vim插件进行注释

```
,cs    # 模块注释（性感注释）  
,cc    # 行注释  
,cu    # 取消注释
```

8. 流程控制

C语言的流程控制分为三种结构：

1. **顺序结构** - 代码从上到下顺序执行
2. **选择结构** - 根据条件选择执行不同的代码
3. **循环结构** - 重复执行某段代码

选择结构

if语句

格式1：单分支

```
if (条件)  
{  
    语句块;  
}
```

格式2：双分支

```
if (条件)  
{  
    条件为真时执行;  
}  
else  
{  
    条件为假时执行;  
}
```

格式3：嵌套if

```
if (条件1)
{
    if (条件2)
    {
        // 条件1和条件2都成立
    }
    else
    {
        // 条件1成立, 条件2不成立
    }
}
else
{
    if (条件3)
    {
        // 条件1不成立, 条件3成立
    }
    else
    {
        // 条件1和条件3都不成立
    }
}
```

格式4: 开关语句 (else if)

```
if (条件1)
{
    语句块1;
}
else if (条件2)
{
    语句块2;
}
else if (条件3)
{
    语句块3;
}
...
else
{
    所有条件都不满足时执行;
}
```

循环结构

for循环

```
for (初始化; 条件; 更新)
{
    循环体;
}
```

while循环

```
初始化;
while (条件)
{
    循环体;
    更新;
}
```

do-while循环

```
do
{
    循环体;
    更新;
} while (条件);    // 注意：这里有分号
```

特点： 至少执行一次循环体

循环类型对比

循环类型	特点	使用场景
for	知道循环次数	循环次数确定
while	先判断后执行	循环次数不确定
do-while	先执行后判断	至少执行一次

break和continue

```
// break: 退出整个循环
for (int i = 0; i < 10; i++)
{
    if (i == 5)
        break;    // 当i=5时退出整个循环
    printf("%d ", i);
}
// 输出: 0 1 2 3 4

// continue: 退出本次循环, 继续下一次
for (int i = 0; i < 10; i++)
{
    if (i == 5)
        continue;    // 当i=5时跳过, 继续下一次
    printf("%d ", i);
}
// 输出: 0 1 2 3 4 6 7 8 9
```

注意： break和continue只对当前所在的循环有效

死循环

```
// 死循环：条件一直满足
while (1)
{
    // 循环体
}

// 死循环注意事项：
// 1. 必须有延迟函数
// 2. 必须有退出条件
```

延迟函数

```
#include <unistd.h>

// sleep - 秒级延迟
sleep(1); // 延迟1秒

// usleep - 微秒级延迟
usleep(1000000); // 延迟1秒 (1000000微秒)
```

随机数

```
#include <stdlib.h>
#include <time.h>

// 获取随机数
int rand(void);

// 设置随机数种子
srand(time(NULL)); // 使用当前时间作为种子
// 或
srand(getpid()); // 使用进程ID作为种子

// 示例：生成1-100的随机数
int num = rand() % 100 + 1;
```

练习：猜数字游戏

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    int answer, guess;
    int count = 0;

    srand(time(NULL));
    answer = rand() % 100 + 1;

    printf("猜数字游戏 (1-100) \n");

    while (1)
    {
        printf("请输入你猜的数字: ");
        scanf("%d", &guess);
        count++;

        if (guess > answer)
            printf("太大了! \n");
        else if (guess < answer)
            printf("太小了! \n");
        else
        {
            printf("恭喜你猜对了! 共猜了%d次\n", count);
            break;
        }
    }

    return 0;
}
```

goto语句

goto 是跳转语句，可以跳转到指定标签位置。


```
// 格式：
goto 标签;

// 示例：goto实现循环
#include <stdio.h>

int main(void)
{
    int i = 0;

loop:
    if (i < 10)
    {
        printf("%d ", i);
        i++;
        goto loop;
    }
    printf("\n");

    return 0;
}
```

goto的使用场景：

1. 深层退出（从多层循环中跳出）
2. 实现循环
3. 容错处理

注意： 尽量少用goto，避免代码逻辑混乱

9. 函数

函数是C语言最小的编程单位，用于封装通用代码，供其他用户调用。

函数分类

按来源分：

- **库函数：** 标准C库提供的功能函数
 - printf, scanf, getchar, putchar
 - rand(), srand()

- 存放位置： `/lib/libc.so.6`
- 自定义函数：用户根据需求实现的函数

函数组成

函数由四部分组成：

```
返回类型 函数名(参数列表)
{
    函数体;
}
```

组成部分	说明
返回类型	char, short, int, long, float, double, long double （共7种） 若无返回值则为 void
函数名	函数的入口地址，命名规则与变量相同
参数列表	类型 参数名，类型 参数名， ... 若无参数则为 void
函数体	{ } 中包含函数功能的实现

有参函数和无参函数

```
// 无参函数
void hello(void)
{
    printf("Hello, World!\n");
}

// 有参函数
int add(int a, int b)
{
    return a + b;
}
```

实参和形参

// 实参 (实际参数): 调用函数时传递的参数
// 形参 (形式参数): 函数定义时的参数

```
int add(int a, int b)    // a, b 是形参
{
    return a + b;
}

int main(void)
{
    int result = add(3, 5);    // 3, 5 是实参
    return 0;
}
```

函数传参方式

值传递: 不会改变原数据

```
void swap_wrong(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
    // 交换的是形参, 实参不变
}
```

地址传递: 会修改原数据

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
    // 通过指针修改实参的值
}

int main(void)
{
    int x = 10, y = 20;
    swap(&x, &y);    // 传递地址
    printf("x=%d, y=%d\n", x, y);    // x=20, y=10
    return 0;
}
```

函数参数传递原则

一般情况：参数个数、类型、顺序要保持一致

特殊情况：不定参函数

不定参函数

```
#include <stdarg.h>

// 函数声明
int add(int count, ...);

// 实现
int add(int count, ...)
{
    va_list ap;          // 声明存储不定参的变量
    int sum = 0;

    va_start(ap, count); // 初始化, count是最后一个有名参数

    for (int i = 0; i < count; i++)
    {
        sum += va_arg(ap, int); // 获取参数
    }

    va_end(ap);          // 销毁

    return sum;
}

// 使用
int main(void)
{
    printf("%d\n", add(2, 12, 34)); // 46
    printf("%d\n", add(3, 12, 34, 56)); // 102
    printf("%d\n", add(5, 12, 34, 56, 78, 90)); // 270
    return 0;
}
```

va_list相关函数：

函数	说明
va_start(ap, last)	初始化, last是最后一个有名参数
va_arg(ap, type)	获取参数, type是参数类型
va_end(ap)	销毁

函数	说明
va_copy(dest, src)	拷贝

递归函数

递归函数是函数直接或间接调用自身。

分类：

- 直接递归：函数调用自身

$$A \Rightarrow A$$

- 间接递归：函数之间相互调用，最终调用自身

$$A \Rightarrow B \Rightarrow \dots \Rightarrow A$$

重要： 使用递归必须有退出条件，否则会导致栈溢出（段错误）

示例：阶乘

```
// 递归实现阶乘
int factorial(int n)
{
    if (n <= 1)           // 退出条件
        return 1;
    return n * factorial(n - 1);    // 递归调用
}

int main(void)
{
    printf("5! = %d\n", factorial(5));    // 120
    return 0;
}
```

示例：斐波那契数列

```
// 递归实现斐波那契数列
int fib(int n)
{
    if (n <= 2)
        return 1;
    return fib(n - 1) + fib(n - 2);
}

int main(void)
{
    for (int i = 1; i <= 10; i++)
    {
        printf("%d ", fib(i));
    }
    printf("\n");
    // 输出: 1 1 2 3 5 8 13 21 34 55
    return 0;
}
```

10. 分文件编程

头文件包含方式

```
// 方式1: <> 从系统目录查找 (/usr/include)
#include <stdio.h>

// 方式2: "" 从当前目录查找, 找不到再从系统目录查找
#include "myheader.h"
```

编译选项: -I 指定头文件路径

```
gcc -I /path/to/headers file.c -o file
```

编译过程

源文件生成可执行文件的4个步骤:

```
gcc file.c -o file    # 一步完成
```

分步编译：

```
# 1. 预处理 (.i 文件)
#     处理伪代码和特殊字符 (#开头的)
#     - 头文件展开
#     - 宏替换
#     - 条件编译
gcc -E file.c -o file.i

# 2. 编译 (.s 文件)
#     检查语法规则和词法错误
gcc -S file.i -o file.s

# 3. 汇编 (.o 文件)
#     汇编代码生成目标代码
gcc -c file.s -o file.o

# 4. 链接
#     链接库文件 (.so .a)
gcc file.o -o file
```

步骤	输出文件	说明
预处理	.i	展开头文件、宏替换
编译	.s	语法检查，生成汇编
汇编	.o	生成目标文件
链接	可执行文件	链接库文件

11. 字符串函数

字符串处理函数

使用字符串函数需要包含头文件： `#include <string.h>`

拷贝函数

```
// strcpy - 字符串拷贝
char *strcpy(char *dest, const char *src);

// strncpy - 拷贝n个字符
char *strncpy(char *dest, const char *src, size_t n);
```

示例：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char dest[128];
    char src[] = "Hello, World!";

    strcpy(dest, src);
    printf("dest: %s\n", dest);    // Hello, World!

    strncpy(dest, src, 5);
    dest[5] = '\0';
    printf("dest: %s\n", dest);    // Hello

    return 0;
}
```

连接函数

```
// strcat - 字符串连接
char *strcat(char *dest, const char *src);

// strncat - 连接n个字符
char *strncat(char *dest, const char *src, size_t n);
```

示例：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char dest[128] = "Hello";
    char src[] = ", World!";

    strcat(dest, src);
    printf("dest: %s\n", dest);    // Hello, World!

    return 0;
}
```

比较函数

```
// strcmp - 字符串比较
int strcmp(const char *s1, const char *s2);
// 返回值: <0 s1<s2, 0 相等, >0 s1>s2

// strncmp - 比较前n个字符
int strncmp(const char *s1, const char *s2, size_t n);
```

示例:

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s1[] = "apple";
    char s2[] = "banana";

    int ret = strcmp(s1, s2);
    if (ret < 0)
        printf("%s < %s\n", s1, s2);
    else if (ret > 0)
        printf("%s > %s\n", s1, s2);
    else
        printf("%s == %s\n", s1, s2);

    return 0;
}
```

查找函数

```
// strchr - 从前向后查找字符
char *strchr(const char *s, int c);

// strrchr - 从后向前查找字符
char *strrchr(const char *s, int c);

// strstr - 查找子串
char *strstr(const char *haystack, const char *needle);

// strcasestr - 不区分大小写查找子串
char *strcasestr(const char *haystack, const char *needle);
```

示例：

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char str[] = "Hello, World!";

    // 查找字符'o'
    char *p = strchr(str, 'o');
    if (p)
        printf("第一次出现'o'的位置: %ld\n", p - str);

    // 从后向前查找
    p = strrchr(str, 'o');
    if (p)
        printf("最后一次出现'o'的位置: %ld\n", p - str);

    // 查找子串
    p = strstr(str, "World");
    if (p)
        printf("找到子串: %s\n", p);

    return 0;
}
```

字符串函数实现示例

```
// 自己实现strcpy
void my_strcpy(char dest[], char src[])
{
    int i = 0;
    while ((dest[i] = src[i++]) != '\0')
        ;
}

// 自己实现strcat
void my_strcat(char dest[], char src[])
{
    int len = strlen(dest);
    int i = 0;
    while ((dest[len + i] = src[i++]) != '\0')
        ;
}

// 自己实现strcmp
int my_strcmp(char s1[], char s2[])
{
    int i = 0;
    while (s1[i] == s2[i] && s1[i] != '\0')
        i++;
    return s1[i] - s2[i];
}
```

12. 内存函数

内存操作函数

memcpy - 内存拷贝

```
void *memcpy(void *dest, const void *src, size_t n);
```

- dest - 目标空间地址
- src - 源空间地址
- n - 拷贝的字节数

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    int src[] = {1, 2, 3, 4, 5};
    int dest[5];

    memcpy(dest, src, sizeof(src));

    for (int i = 0; i < 5; i++)
        printf("%d ", dest[i]);
    printf("\n");    // 1 2 3 4 5

    return 0;
}
```

memmove - 内存移动

```
void *memmove(void *dest, const void *src, size_t n);
```

与memcpy的区别:

- 如果源和目标内存空间**不重叠**，两者效果一样
- 如果源和目标内存空间**重叠**，必须使用memmove

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    int arr[] = {1, 2, 3, 4, 5, 6, 7, 8};

    // 重叠情况: 将arr[2..6]移动到arr[0..4]
    memmove(arr, arr + 2, 5 * sizeof(int));

    for (int i = 0; i < 8; i++)
        printf("%d ", arr[i]);
    printf("\n");    // 3 4 5 6 7 4 5 6

    return 0;
}
```

memset - 内存设置

```
void *memset(void *s, int c, size_t n);
```

- s - 要设置的内存地址
- c - 设置的值
- n - 设置的字节数

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    int arr[10];

    // 将数组所有元素设置为0
    memset(arr, 0, sizeof(arr));

    // 将字符串初始化为空格
    char str[20];
    memset(str, ' ', sizeof(str) - 1);
    str[sizeof(str) - 1] = '\\0';

    return 0;
}
```

13. 数组进阶

二维数组

```
// 定义格式
type name[行数][列数];

// 初始化
int arr[2][3] = {1, 2, 3, 4, 5, 6};
int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
```


二维数组遍历

```
#include <stdio.h>

#define ROW 2
#define COL 3

int main(void)
{
    int arr[ROW][COL] = {1, 2, 3, 4, 5, 6};

    for (int i = 0; i < ROW; i++)
    {
        for (int j = 0; j < COL; j++)
        {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

数组sizeof计算

```
#include <stdio.h>

#define ROW 5
#define COL 8

void test(int arr[][COL])
{
    printf("sizeof(arr): %lu\n", sizeof(arr));          // 指针大小 (4或8)
    printf("sizeof(arr[0]): %lu\n", sizeof(arr[0]));    // 一行的大小
    printf("sizeof(arr[0][0]): %lu\n", sizeof(arr[0][0])); // 一个元素的大小

    // 计算行数和列数
    printf("row: %lu\n", sizeof(arr) / sizeof(arr[0])); // 错误! 指针/一行
    printf("col: %lu\n", sizeof(arr[0]) / sizeof(arr[0][0])); // 正确
}

int main(void)
{
    int arr[ROW][COL];

    printf("====main =====\n");
    printf("sizeof(arr): %lu\n", sizeof(arr));          // 5*8*4 = 160
    printf("sizeof(arr[0]): %lu\n", sizeof(arr[0]));    // 8*4 = 32
    printf("sizeof(arr[0][0]): %lu\n", sizeof(arr[0][0])); // 4

    // 计算行列数
    printf("row: %lu\n", sizeof(arr) / sizeof(arr[0])); // 5
    printf("col: %lu\n", sizeof(arr[0]) / sizeof(arr[0][0])); // 8

    printf("====test =====\n");
    test(arr);
    return 0;
}
```

数组地址偏移

```
#include <stdio.h>

int main(void)
{
    int arr[2][3] = {1, 2, 3, 4, 5, 6};

    // &arr: 整个数组的首地址, +1偏移整个数组大小
    printf("&arr: %p, &arr+1: %p\n", &arr, &arr + 1);

    // arr: 行的首地址, +1偏移一行的大小
    printf("arr: %p, arr+1: %p\n", arr, arr + 1);

    // arr[0]: 列的首地址, +1偏移一列的大小
    printf("arr[0]: %p, arr[0]+1: %p\n", arr[0], arr[0] + 1);

    // &arr[0][0]: 第一个元素的地址, +1偏移一个元素的大小
    printf("&arr[0][0]: %p, &arr[0][0]+1: %p\n", &arr[0][0], &arr[0][0] + 1);

    return 0;
}
```

fgets获取字符串

```
#include <stdio.h>

int main(void)
{
    char buf[1024];

    printf("请输入字符串: ");
    fgets(buf, sizeof(buf), stdin);

    // 去除末尾的换行符
    if (buf[strlen(buf) - 1] == '\n')
        buf[strlen(buf) - 1] = '\0';

    printf("你输入的是: %s\n", buf);

    return 0;
}
```

14. 头文件防重复包含

问题

当头文件被多次包含时，会导致重复定义错误。

解决方案

使用条件编译宏：

```
#ifndef __FILENAME_H__
#define __FILENAME_H__

// 头文件内容
#include <stdio.h>
#include <stdlib.h>

// 函数声明
void hello(void);

#endif // __FILENAME_H__
```

说明：

- `#ifndef` - 如果没有定义宏
- `#define` - 定义宏
- `#endif` - 结束条件编译

这样即使头文件被多次包含，也只会生效一次。

15. Makefile详解

Makefile简介

Makefile用于多文件编译，是一门规则性的语言。

基本格式

规则名: 依赖
 (tab) 命令

示例：

```
all:
    echo "Hello, Makefile"

clean:
    rm -f *.o
```

变量

```
# 定义变量
CC = gcc
TARGET = hello
OBJS = main.o func.o

# 取值
${TARGET} 或 $(TARGET)

# 赋值方式
VAL1 = abc          # 赋值
VAL2 += def          # 追加
VAL3 := abc          # 立即赋值（只能取前面定义的变量）
VAL4 ?= abc          # 如果未定义则赋值
```

依赖关系

```
# 规则依赖其他规则
all: install clean
    echo "all"

install:
    echo "install"

clean:
    echo "clean"
```

执行顺序：先执行依赖（install、clean），再执行all

伪指令.PHONY

当规则名和文件名同名时，使用.PHONY指定伪目标：

```
.PHONY: clean

clean:
    rm -f *.o
```

自动变量

变量	说明
<code>\$@</code>	规则名
<code>\$^</code>	所有依赖（去重）
<code>\$+</code>	所有依赖（保留重复）
<code>\$<</code>	第一个依赖

示例：

```
CC = gcc
TARGET = hello
OBJS = main.o func.o

${TARGET}:${OBJS}
    ${CC} $^ -o $@

# 模式规则
%.o:%.c
    ${CC} -c $< -o $@
```

Makefile命名

- 默认文件名：Makefile 或 makefile
- 其他命名：使用 `-f` 指定

```
make -f MyMakefile
```

16. 动态库和静态库

库文件简介

- 系统库路径： `/lib`
- 动态库： `.so` （Shared Object）
- 静态库： `.a` （Archive）

动态库创建和使用

步骤1：编译生成.o文件

```
gcc -c -fPIC func1.c -o func1.o
gcc -c -fPIC func2.c -o func2.o
```

注意：64位系统需要 `-fPIC` 参数

步骤2：生成动态库

```
# 格式: lib + 库名 + .so
gcc -shared -fPIC -o libmyfunc.so func1.o func2.o
```

- `-shared` - 生成动态共享库
- `-fPIC` - 生成与位置无关的代码

步骤3：编译主程序

```
gcc main.c -I . -L . -lmyfunc -o hello
```

- `-I .` - 指定头文件路径
- `-L .` - 指定库文件路径
- `-lmyfunc` - 指定链接的库名（[去掉lib和.so](#)）

步骤4：运行

```
# 方式1：移动库到系统目录
cp libmyfunc.so /lib/
```

```
# 方式2：配置库路径
echo "/path/to/lib" >> /etc/ld.so.conf
ldconfig
```


静态库创建和使用

步骤1：编译生成.o文件

```
gcc -c func1.c -o func1.o
gcc -c func2.c -o func2.o
```

步骤2：生成静态库

```
# 格式: lib + 库名 + .a
ar -cr libmyfunc.a func1.o func2.o
```

- -c - 创建库
- -r - 添加模块到库中

步骤3：编译主程序

```
gcc main.c -L . -lmyfunc -o hello
```

步骤4：运行

```
./hello      # 直接运行，不需要库文件
```

动态库和静态库区别

特性	动态库(.so)	静态库(.a)
加载时机	运行时加载	编译时加载
文件大小	较小（共享）	较大（包含所有代码）
执行效率	较低（需要动态链接）	较高
依赖性	需要库文件存在	不需要库文件
优先级	高	低

头文件示例

```
#ifndef __FUNC_H__
#define __FUNC_H__

#include <stdio.h>
#include <stdlib.h>

#define PI 3.14159

// 函数声明
void hello(void);
int add(int a, int b);

#endif // __FUNC_H__
```

Makefile示例（多文件编译）

```
CC = gcc
TARGET = hello
OBSJS = main.o func1.o func2.o

${TARGET}:${OBSJS}
    ${CC} $^ -o $@

# 模式规则
%.o:%.c
    ${CC} -c $< -o $@

clean:
    rm -f ${TARGET} *.o

.PHONY: clean
```

17. 指针

指针是C语言的精华所在，也是难点。

指针的定义

```
// 格式：基类型 *指针名 = 地址；
int *p = NULL;    // 定义整型指针，初始化为NULL
int a = 10;
p = &a;           // p指向a的地址

printf("a = %d\n", a);           // 10
printf("*p = %d\n", *p);         // 10, 通过指针访问
printf("&a = %p\n", &a);         // a的地址
printf("p = %p\n", p);          // 同上
```

指针的核心概念

- & - 取地址运算符
- * - 解引用运算符（间接访问）
- 指针本身也是一个变量，存储的是地址值

动态内存申请

malloc - 申请堆内存：

```
#include <stdlib.h>

// 申请空间
void *malloc(size_t size);

// 带计数器的申请
void *calloc(size_t nmemb, size_t size);

// 重新调整大小
void *realloc(void *ptr, size_t size);

// 释放空间
void free(void *ptr);
```

示例：

```

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // 申请10个整数的空间
    int *arr = (int *)malloc(10 * sizeof(int));
    if (arr == NULL)
    {
        printf("内存申请失败! \n");
        return -1;
    }

    // 使用
    for (int i = 0; i < 10; i++)
        arr[i] = i + 1;

    // 释放
    free(arr);
    arr = NULL;    // 避免野指针

    return 0;
}

```

注意事项:

- 申请和释放要一一对应，否则导致内存泄漏
- free 后建议将指针置为 NULL
- malloc 申请的内存存在 **堆(heap)** 中

void* 万能指针

void * 可以接受任意类型的指针，用于泛型编程。

```

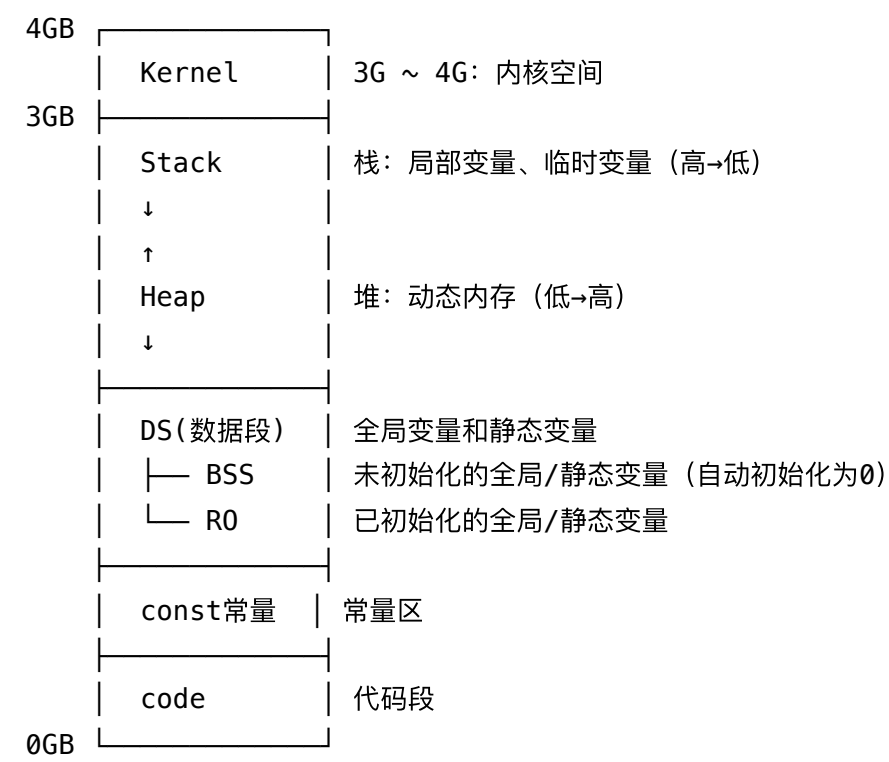
void *p;
int a = 10;
char c = 'A';

p = &a;    // 可以指向int
p = &c;    // 也可以指向char

```

18. 内存布局

32位系统程序内存空间（4GB）



各区域特点

区域	存放内容	特点
栈(stack)	局部变量、函数参数	由高到低分配，函数调用时分配
堆(heap)	malloc/calloc/realloc	由低到高分配，需要手动free
数据段(DS)	全局变量、静态变量	程序启动前初始化
BSS	未初始化全局/静态变量	自动赋值为0
RO	已初始化全局/静态变量	初始化后的值
常量区	const常量	只读
代码段(code)	程序指令	只读

栈与堆的对比

```
#include <stdio.h>
#include <stdlib.h>

int global_a;          // BSS段, 自动初始化为0
int global_b = 10;     // R0数据段

void test(void)
{
    static int count = 0; // 数据段, 程序启动前初始化为0
    int local = 10;       // 栈
    char *p = (char *)malloc(100); // p在栈, 指向的空间在堆
    free(p);
}
```

19. 修饰符

static

static修饰有三个作用:

```
// 1. 修饰局部变量: 延长生存周期到整个程序
void test(void)
{
    static int count = 0; // 只初始化一次, 值在函数调用间保持
    count++;
    printf("%d\n", count);
}

// 2. 修饰全局变量: 限制作用域为当前文件
static int file_var = 10; // 只在本文件有效

// 3. 修饰函数: 限制函数只能在本文件中调用
static void helper(void) // 只在本文件有效
{
    printf("helper\n");
}
```

const

const修饰的变量为只读，不可修改：

```
const int a = 10;
// a = 20;    // 错误！不能修改const变量

// const修饰指针
const int *p1;    // p1指向的内容不可改，p1可改
int * const p2;    // p2不可改，p2指向的内容可改
const int * const p3; // 都不可改
```

volatile

告诉编译器不要优化该变量，每次从内存中读取：

```
volatile int flag = 0;
// 常用于：硬件寄存器、多线程共享变量、信号处理
```

register

建议将变量存放到寄存器中，提高访问效率：

```
register int i;
for (i = 0; i < 1000000; i++)
    ;
```

extern

声明外部变量或函数，用于跨文件引用：

```
extern int global_var;    // 声明在其他文件中定义的全局变量
extern void func(void);    // 声明在其他文件中定义的函数
```

20. 多级指针

二级指针

```
// 格式: type **name = value;

int a = 10;
int *q = &a;          // 一级指针
int **p = &q;          // 二级指针, 存储一级指针的地址

printf("a = %d\n", a);      // 10
printf("*q = %d\n", *q);    // 10
printf("**p = %d\n", **p);  // 10

// 修改值
**p = 123;
printf("a = %d\n", a);      // 123
```

多级指针规律

```
int *****p = NULL;  // 8级指针

// 解引用层级递减:
*****p    // 0级 (p本身)
*p          // 7级指针
**p         // 6级指针
...
*****p     // 2级指针
```

数组指针

数组指针：首先是指针，再是数组，说明这个指针指向的是一个**数组**。


```
// 格式: type (*name)[列数];
int arr[2][3] = {1, 2, 3, 4, 5, 6};
int (*p)[3] = NULL; // 数组指针, 指向每行有3个元素的数组

p = arr; // p指向arr的第一行

// 访问方式 (多种等价)
printf("%d ", p[i][j]); // 最直观
printf("%d ", (*(p + i) + j));
printf("%d ", *(p[i] + j));
printf("%d ", (*(p + i))[j]);
```

注意： 多维数组和多级指针没有关系！

概念	说明	等价关系
一级指针	int *p	完全等价一维数组
数组指针	int (*p)[N]	等价多维数组
指针数组	int *p[N]	数组的每个元素是指针

指针数组

指针数组：首先是数组，再是指针，说明数组的每个成员都是**指针**。

```
// 格式: type *name[元素个数];

int a = 1, b = 2, c = 3;
int *arr[3] = {&a, &b, &c};

for (int i = 0; i < 3; i++)
    printf("%d ", *arr[i]); // 1 2 3
```

指针作为函数参数

指针作为函数参数可以**修改外部变量的值**：

```

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

// void * 可以接受任意类型的指针
void print_value(void *p, char type)
{
    if (type == 'i')
        printf("%d\n", *(int *)p);
    else if (type == 'c')
        printf("%c\n", *(char *)p);
}

```

注意： 不可以返回作用域在本函数内部且非静态局部变量的地址。

一级指针 vs 一维数组

```

int arr[10] = {1, 2, 3, 4, 5};
int *p = arr;

// 以下访问方式等价
printf("%d ", arr[i]);
printf("%d ", p[i]);
printf("%d ", *(arr + i));
printf("%d ", *(p + i));

```

21. VT码与终端控制

VT码用于对终端进行简单控制，如表单定位、颜色、清屏等。

ANSI控制码速查

光标控制：

控制码	说明
\033[X;YH	将光标移至第X行第Y列

控制码	说明
\033[nA	光标上移n列
\033[nB	光标下移n列
\033[nC	光标右移n行
\033[nD	光标左移n行
\033[s	保存光标位置
\033[u	恢复光标位置
\033[?25l	隐藏光标
\033[?25h	显示光标

屏幕控制：

控制码	说明
\033[2J	清屏（相当于cls）
\033[K	清除光标到行尾

字符属性：

控制码	说明
\033[0m	恢复默认
\033[1m	高亮/加粗
\033[4m	下划线
\033[5m	闪烁
\033[7m	反相显示

颜色代码：

颜色	前景	背景
黑	30	40

颜色	前景	背景
红	31	41
绿	32	42
黄	33	43
蓝	34	44
紫红	35	45
青蓝	36	46
白	37	47

VT码使用示例

```
#include <stdio.h>

int main(void)
{
    // 清屏
    printf("\033[2J");

    // 在第10行第20列显示红色文字
    printf("\033[10;20H");
    printf("\033[31mHello, World!\033[0m");

    // 在(5,10)用蓝底白字画一个字符
    printf("\033[5;10H");
    printf("\033[37;44mX\033[0m");

    // 隐藏光标
    printf("\033[?25l");

    // 显示光标
    printf("\033[?25h");

    return 0;
}
```

VT码函数封装

```
// vt.h
#ifndef __VT_H__
#define __VT_H__

// 定位光标
void gotoxy(int x, int y);
// 在某位置输出字符
void putchar(int x, int y, int fcolor, int bcolor, char ch);
// 在某位置输出字符串
void_putstr(int x, int y, int fcolor, int bcolor, char *s);
// 画水平线
void draw_horizontal(int x, int y, int fcolor, int bcolor, int len, char ch);
// 画垂直线
void draw_vertical(int x, int y, int fcolor, int bcolor, int len, char ch);
// 画矩形框
void draw_box(int x, int y, int fcolor, int bcolor, int h, int w, char ch);

#endif // __VT_H__
```

键盘输入（read）

使用 read 系统调用直接读取键盘输入：

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    char key[8];
    int ret;

    // 关闭回显
    system("stty -echo");
    // 关闭缓冲区
    system("stty -icanon");

    system("clear");

    while (1)
    {
        ret = read(0, key, sizeof(key));

        if (key[0] == 'q')            // 按q退出
            break;
        else if (key[0] == 'w')       // 上
            printf("\033[A");
        else if (key[0] == 's')       // 下
            printf("\033[B");
        else if (key[0] == 'a')       // 左
            printf("\033[D");
        else if (key[0] == 'd')       // 右
            printf("\033[C");

        memset(key, 0, sizeof(key));
        fflush(NULL);
    }

    // 恢复回显和缓冲区
    system("stty echo");
    system("stty icanon");

    return 0;
}
```

read函数：

```
ssize_t read(int fd, void *buf, size_t count);  
// fd=0 表示键盘标准输入  
// buf 存储数据空间地址  
// count 存储空间大小  
// 返回值：成功获取键值个数，失败返回-1
```

22. 指针函数与函数指针

指针函数

指针函数：首先是**函数**，再是指针，说明函数的**返回类型是指针**。

```
// 格式：type *func_name(args)  
  
int *get_value(void)  
{  
    static int a = 10;    // 必须用static  
    return &a;            // 返回静态变量的地址  
}  
  
int main(void)  
{  
    int *p = get_value();  
    printf("*p = %d\n", *p); // 10  
    return 0;  
}
```

注意： 不可以返回非静态局部变量的地址，因为函数返回后栈空间被回收。

函数指针

函数指针：首先是**指针**，再是函数，说明这个指针指向**函数名（函数入口地址）**。

定义格式：

```
// 格式: return_type (*pointer_name)(参数类型列表);
```

```
void (*p)(void) = NULL;           // 指向返回void、无参的函数  
int (*q)(int, int) = NULL;        // 指向返回int、两个int参数的函数
```

使用示例:

```
#include <stdio.h>  
  
void hello(void)  
{  
    printf("Hello!\n");  
}  
  
int add(int a, int b)  
{  
    return a + b;  
}  
  
int main(void)  
{  
    void (*p)(void) = NULL;  
    int (*q)(int, int) = NULL;  
  
    p = hello;           // 函数名就是函数地址  
    p();                 // 通过函数指针调用  
  
    q = add;  
    printf("add(2, 3) = %d\n", q(2, 3));    // 5  
  
    return 0;  
}
```

函数指针要求三一致原则:

1. 返回类型一致
2. 参数类型一致
3. 参数个数一致

typedef定义函数指针类型

```
// 用typedef简化函数指针
typedef int (*FUNC)(int, int);

FUNC p = add;           // 等价于 int (*p)(int, int) = add;
printf("%d\n", p(5, 3));

// 函数指针作为参数
void calculate(int a, int b, FUNC func)
{
    printf("result: %d\n", func(a, b));
}

int main(void)
{
    calculate(5, 3, add);
    calculate(5, 3, sub);
    return 0;
}
```

函数指针数组

```
#include <stdio.h>

typedef int (FUNC)(int, int);

int add(int a, int b) { return a + b; }
int sub(int a, int b) { return a - b; }
int mul(int a, int b) { return a * b; }
int dev(int a, int b) { return a / b; }
int mod(int a, int b) { return a % b; }

int main(void)
{
    // 函数指针数组
    FUNC *p[5] = {add, sub, mul, dev, mod};
    char op[] = {'+', '-', '*', '/', '%'};

    // 遍历调用
    for (int i = 0; i < 5; i++)
    {
        printf("%d %c %d = %d\n", 10, op[i], 3, p[i](10, 3));
    }

    return 0;
}
```

23. 结构体

结构体是**自定义类型**，把不同的数据类型保存到同一片内存空间中。

结构体定义

```
// 定义结构体类型
struct struct_name {
    type member1;
    type member2;
    // ...
};
```

```
// 示例：学生结构体
struct cls_t {
    char name[64];
    int age;
    float score;
};
```

结构体变量定义和初始化

```
// 定义并初始化结构体变量
struct cls_t cls = {"tom", 18, 99.88};

// 访问成员：. 和 ->
// 非指针变量：用 .
printf("name: %s\n", cls.name);
printf("age: %d\n", cls.age);

// 指针变量：用 ->
struct cls_t *p = &cls;
printf("name: %s\n", p->name);
printf("age: %d\n", p->age);
```

结构体数组

```
#define MAX 10

struct cls_t cls[MAX] = {
    {"tom", 1001, 18, 'M', 88.5},
    {"jim", 1002, 19, 'M', 99.0},
    {"mary", 1003, 17, 'F', 66.7}
};

// 遍历:
for (int i = 0; i < MAX; i++)
{
    printf("name: %s | age: %d | score: %.2f\n",
        cls[i].name, cls[i].age, cls[i].score);
    // 或 cls[i].name 等价于 (cls + i)->name
}
```

结构体指针

```
// 普通方式 (栈上分配)
struct cls_t cls;
struct cls_t *p = &cls;          // p指向栈上的结构体

// 动态申请 (堆上分配)
struct cls_t *p = (struct cls_t *)malloc(sizeof(struct cls_t));
if (p == NULL)
    return -1;

strcpy(p->name, "tom");
p->age = 18;

free(p);          // 别忘了释放
```

柔性数组

柔性数组是结构体最后一个成员，不占结构体空间，用于动态变长数据：

```

struct cls_t {
    int age;
    float score;
    char name[];    // 柔性数组（必须是最后一个成员）
};

// 特点：
// 1. 只能在结构体中定义，且必须是最后一个成员
// 2. 不占结构体内存空间
// 3. 内存连续分配，访问效率更高，释放简单

// 申请空间
struct cls_t *p = (struct cls_t *)malloc(sizeof(struct cls_t) + 10);
strcpy(p->name, "tom");

// 只释放一次即可
free(p);

```

申请空间大小 = 结构体空间大小 + 柔性数组需要的空间大小

结构体大小与对齐

```

struct op_t {
    char ch;    // 1字节
    int a;      // 4字节
};

// sizeof(struct op_t) = 8 (不是5)
// 原因：内存对齐

// 32位系统默认最大对齐是4字节
// 成员小于4字节时，以最大成员大小对齐

```

手动设置对齐方式：

```
#pragma pack(1)          // 1字节对齐（不填充）
struct op_t {
    char ch;
    int a;
};
#pragma pack(0)          // 恢复默认
// 现在 sizeof(struct op_t) = 5
```

结构体位域（位操作）

```
struct op_t {
    unsigned int a : 2;    // 只使用2位
    char ch : 3;          // 只使用3位
};
// a 的取值范围：0~3（2位）
```

结构体作为函数参数

```
// 值传递（不会修改原数据，但会拷贝整个结构体）
void print_info(struct cls_t cls)
{
    printf("name: %s\n", cls.name);
}

// 地址传递（不会拷贝，可以修改原数据）—— 推荐
void modify_info(struct cls_t *cls)
{
    cls->age = 20;
    strcpy(cls->name, "new_name");
}
```

结构体嵌套

结构体不可以自己嵌套自己，但可以**联合**其他结构体：

```
struct CC { int c; };
struct BB {
    int b;
    struct CC c;        // 联合（包含其他结构体）
};
struct AA {
    int a;
    struct BB b;        // 联合
};

// 访问
struct AA a = {123, {789, 1122}};
printf("%d\n", a.a);    // 123
printf("%d\n", a.b.b);  // 789
printf("%d\n", a.b.c.c); // 1122
```

24. 共用体与枚举

共用体（union）

共用体的所有成员共用同一片内存空间：

```

// 定义共用体
union op_t {
    char ch;
    int a;
};

int main(void)
{
    union op_t op;

    // 大小由成员中最大者决定
    printf("sizeof(union op_t) = %lu\n", sizeof(union op_t)); // 4

    // 所有成员地址相同
    printf("&op.ch = %p\n", &op.ch);
    printf("&op.a = %p\n", &op.a); // 地址相同!

    op.ch = 'A';
    printf("op.a = %d\n", op.a); // 65 (被覆盖)

    return 0;
}

```

共用体特点：

- 所有成员共用同一片内存
- 大小 = 最大成员的大小
- 任一时刻只有一个成员有效
- 修改一个成员会影响其他成员

练习：检测大小端模式


```

#include <stdio.h>

union check_t {
    int a;
    char ch;
};

int main(void)
{
    union check_t c;
    c.a = 1;

    if (c.ch == 1)
        printf("小端模式\n");    // 低字节存低地址
    else
        printf("大端模式\n");    // 低字节存高地址

    return 0;
}

```

枚举 (enum)

枚举是一组命名整型常量的集合：

```

// 定义枚举
enum Color { RED, GREEN = 5, BLUE };

int main(void)
{
    enum Color c = RED;

    printf("RED = %d\n", RED);    // 0 (默认从0开始)
    printf("GREEN = %d\n", GREEN); // 5 (手动指定)
    printf("BLUE = %d\n", BLUE);  // 6 (自动+1)

    return 0;
}

```

枚举规则：

- 默认第一个值为0，后续自动加1

- 可以手动指定值，后续继续自动加1

25. 预处理

预处理处理的是伪代码（# 开头）和特殊字符，包括：头文件、宏命令、条件编译。

宏命令（#define）

宏命令是纯文本替换，在预处理阶段展开。

```
// 定义宏常量
#define PI 3.14
#define R 2

// 带参数的宏
#define S(r) (PI * (r) * (r))
#define ADD(a, b) ((a) + (b))

// 使用
printf("PI = %f\n", PI);
printf("S = %f\n", S(3));
printf("add = %d\n", ADD(2, 3));
```

宏 vs 函数

特性	宏	函数
处理时机	编译时（预处理阶段）	运行时调用
执行效率	高（无调用开销）	较低（有调用开销）
空间效率	低（每次展开一份）	高（只有一份代码）
安全性	较低（需注意运算符优先级）	高
通用性	高（不需要类型）	较低（参数需明确类型）
代码量	一行实现	多行实现

选择原则： 函数代码少于5行，可以考虑用宏实现。

#运算符（字符串连接符）

将宏参数转换为字符串：

```
#define PRI(a) printf(#a " : %d\n", a)

int result = 99;
PRI(result);    // 展开为: printf("result " : %d\n", result);
// 输出: result : 99
```

##运算符（字符连接符）

将两个标记连接成一个：

```
#define TEST(a, b) a##b

int TEST(x, y) = 123;    // 等价于 int xy = 123;
printf("xy = %d\n", xy);
```

带返回值的宏

使用（{...}）语句块实现带返回值的宏：

```
#define MAX(a, b) ({typeof(a) max; \
    if ((a) > (b)) \
        max = (a); \
    else \
        max = (b); \
    max;})

// typeof() 获取变量的类型

int m = MAX(10, 20);    // 整型比较
float f = MAX(10.5, 2.8); // 浮点型比较（通用!）
```

容错宏（ERRP）

使用goto实现错误处理跳转，用 do{}while(0) 包装确保安全性：

```

#define ERRP(con, info, ret) do{                                \
    if (con)                                                    \
    {                                                            \
        printf(#info " error Line:%d File:%s\n", \
            __LINE__, __FILE__);    \
        ret;                                                    \
    }                                                            \
}while(0)

```

// 使用示例

```

int ***p = NULL;
p = (int ***)malloc(sizeof(int **));
ERRP(NULL == p, malloc p, goto ERR1);

*p = (int **)malloc(sizeof(int *));
ERRP(NULL == *p, malloc *p, goto ERR2);

free(*p);
free(p);
return 0;
ERR2:
    free(p);
ERR1:
    return -1;

```

条件编译

条件编译允许根据条件选择性编译代码。

格式1: #if 常量

```

#if 1
    printf("hello!\n");    // 1为真, 执行
#endif

```

格式2: #if 宏名

```
#define FLAG 1

#if FLAG
    printf("world!\n");    // FLAG非0, 执行
#endif
```

格式3: #if-#elif-#else

```
#define FLAG 2

#if FLAG == 1
    printf("AA!\n");
#elif FLAG == 2
    printf("BB!\n");    // FLAG=2, 执行这里
#else
    printf("other!\n");
#endif
```

格式4: #ifdef / #ifndef

```
#define AA

#ifdef AA
    printf("定义了 AA\n");    // 如果定义了则执行
#endif

#ifndef BB
    printf("未定义 BB\n");    // 如果未定义则执行
#endif
```

取消宏: #undef

```
#undef AA    // 取消宏定义
```

特殊字符

```
#include <stdio.h>

void test(void)
{
    printf("test: line=%d func=%s file=%s\n",
        __LINE__, __func__, __FILE__);
}

int main(void)
{
    printf("line: %d\n", __LINE__);        // 当前行号
    printf("func: %s\n", __func__);        // 当前函数名
    printf("file: %s\n", __FILE__);        // 当前文件名
    printf("time: %s\n", __TIME__);        // 编译时间
    printf("date: %s\n", __DATE__);        // 编译日期

    return 0;
}
```

特殊字符	含义
__LINE__	当前行号
__FILE__	当前文件名
__func__	当前函数名
__TIME__	编译时间
__DATE__	编译日期

26. 文件IO

打开文件 fopen

```
#include <stdio.h>

FILE *fopen(const char *path, const char *mode);
// 成功：返回文件指针
// 失败：返回 NULL
```

文件打开模式：

模式	说明	文件不存在	文件存在
"r"	只读	失败(返回NULL)	从开头读
"r+"	可读可写	失败	覆盖方式写
"w"	只写	创建文件	清空后写
"w+"	可读可写	创建	清空
"a"	追加写	创建	从末尾写
"a+"	可读可写追加	创建	读从开头，写从末尾

关闭文件 fclose

```
int fclose(FILE *fp);
```

字符读写（fgetc / fputc）

```
// 读取一个字符
int fgetc(FILE *stream);
// 返回：读取的字符（成功），EOF（-1）（失败/文件结束）

// 写入一个字符
int fputc(int c, FILE *stream);
```

示例：实现cp（文件复制）

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    FILE *fpr = NULL, *fpw = NULL;
    int ch;

    if (argc != 3)
    {
        printf("用法: %s <源文件> <目标文件>\n", argv[0]);
        return -1;
    }

    fpr = fopen(argv[1], "r");
    if (fpr == NULL)
    {
        printf("打开源文件失败!\n");
        return -1;
    }

    fpw = fopen(argv[2], "w");
    if (fpw == NULL)
    {
        fclose(fpr);
        printf("打开目标文件失败!\n");
        return -1;
    }

    while (1)
    {
        ch = fgetc(fpr);
        if (ch == EOF)
            break;
        fputc(ch, fpw);
    }

    fclose(fpw);
    fclose(fpr);

    return 0;
}
```


行读写 (fgets / fputs)

```
// 读取一行
char *fgets(char *s, int size, FILE *stream);
// 返回: s (成功), NULL (失败/结束)
```

```
// 写入一行
int fputs(const char *s, FILE *stream);
```

示例:

```
#include <stdio.h>

int main(void)
{
    FILE *fp = NULL;
    char buf[1024];

    // 写入
    fp = fopen("test.txt", "w");
    if (fp)
    {
        fputs("Hello, World!\n", fp);
        fputs("This is line 2.\n", fp);
        fclose(fp);
    }

    // 读取
    fp = fopen("test.txt", "r");
    if (fp)
    {
        while (fgets(buf, sizeof(buf), fp) != NULL)
        {
            printf("%s", buf);
        }
        fclose(fp);
    }

    return 0;
}
```

使用容错宏的文件操作

结合容错宏，文件操作的错误处理更简洁：

```
#include <stdio.h>
#include <stdlib.h>
#include "share.h"

int main(int argc, char *argv[])
{
    FILE *fpr = NULL, *fpw = NULL;
    int ch;

    ERRP(NULL == (fpr = fopen(argv[1], "r")), fopen read, goto ERR1);
    ERRP(NULL == (fpw = fopen(argv[2], "w")), fopen write, goto ERR2);

    while (1)
    {
        ch = fgetc(fpr);
        if (ch == EOF) break;
        ERRP(EOF == fputc(ch, fpw), fputc, goto ERR3);
    }

    fclose(fpw);
    fclose(fpr);
    return 0;
ERR3:
    fclose(fpw);
ERR2:
    fclose(fpr);
ERR1:
    return -1;
}
```

27. 文件IO进阶

按块读写 fread / fwrite

fread和fwrite用于二进制文件的读写，效率比fgetc/fputc高。

```
// 读
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
// ptr - 数据存储空间地址
// size - 每个数据块的大小
// nmemb - 数据块个数
// 返回: 成功读取的数据块个数 (应等于nmemb)

// 写
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
// 返回: 成功写入的数据块个数
```

示例：用fread/fwrite实现cp

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    FILE *fpr = NULL, *fpw = NULL;
    char buf[1024];
    int n;

    fpr = fopen(argv[1], "r");
    fpw = fopen(argv[2], "w");

    while (1)
    {
        n = fread(buf, 1, sizeof(buf), fpr);
        if (n <= 0) break;
        fwrite(buf, 1, n, fpw);
    }

    fclose(fpw);
    fclose(fpr);
    return 0;
}
```

说明： 操作二进制文件使用fread/fwrite，文本文件可用fgetc/fputc。

按格式读写 fprintf / fscanf

```
// 格式化写入
int fprintf(FILE *stream, const char *format, ...);

// 格式化读取
int fscanf(FILE *stream, const char *format, ...);
```

示例：结构体读写到文件

```
#include <stdio.h>

struct cls_t {
    char name[64];
    int id;
    int age;
    char sex;
    float score;
};

// 写入学生信息到文件
void save_student(FILE *fp, struct cls_t *cls)
{
    fprintf(fp, "%s %d %d %c %.2f\n",
            cls->name, cls->id, cls->age, cls->sex, cls->score);
}

// 从文件读取学生信息
void load_student(FILE *fp, struct cls_t *cls)
{
    fscanf(fp, "%s %d %d %c %f",
            cls->name, &cls->id, &cls->age, &cls->sex, &cls->score);
}
```

文件指针定位 fseek / ftell / rewind

```
// 定位文件指针
int fseek(FILE *stream, long offset, int whence);
// offset - 偏移量
// whence - 偏移参考位置
// 返回：成功0，失败-1

// 获取当前文件指针位置
long ftell(FILE *stream);

// 重置文件指针到开头
void rewind(FILE *stream);
```

偏移参考位置：

宏	含义
SEEK_SET	文件开头
SEEK_CUR	当前位置
SEEK_END	文件末尾

示例：

```
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("test.txt", "r");
    char ch;

    // 读取文件全部内容 (第一遍)
    while ((ch = fgetc(fp)) != EOF)
        putchar(ch);

    // 回到文件开头
    fseek(fp, 0, SEEK_SET);

    // 再次读取 (第二遍)
    while ((ch = fgetc(fp)) != EOF)
        putchar(ch);

    // 获取文件大小
    fseek(fp, 0, SEEK_END);
    printf("文件大小: %ld 字节\n", ftell(fp));

    // 从末尾向前偏移1字节
    fseek(fp, -1, SEEK_END);
    ch = fgetc(fp);
    printf("最后一个字符: %c\n", ch);

    fclose(fp);
    return 0;
}
```

grep命令实现

```
#include <stdio.h>
#include <string.h>

// 用法: ./grep keyword filename
int main(int argc, char *argv[])
{
    FILE *fp = NULL;
    char buf[1024];
    char *p = NULL, *s = NULL;
    char tmp[128];
    int line = 0;

    if (argc != 3)
    {
        printf("用法: %s keyword filename\n", argv[0]);
        return -1;
    }

    fp = fopen(argv[2], "r");
    if (fp == NULL)
        return -1;

    while (fgets(buf, sizeof(buf), fp) != NULL)
    {
        line++;
        if ((p = strstr(buf, argv[1])) != NULL)
        {
            printf("%d: ", line);
            // 关键字红色高亮
            while ((s = strstr(p, argv[1])) != NULL)
            {
                memset(tmp, 0, sizeof(tmp));
                memmove(tmp, p, s - p);
                printf("%s", tmp);
                printf("\033[31m%s\033[0m", argv[1]);
                p = s + strlen(argv[1]);
            }
            printf("%s", p);
        }
    }
}
```

```
fclose(fp);  
return 0;  
}
```

28. 动态数组（数据结构基础）

动态数组解决了静态数组空间固定（不足或浪费）的问题，是正式学习数据结构前的重要过渡。

思路

1. 初始分配较小空间
2. 数据满时重新申请更大的空间（原空间 $\times 1.5$ ）
3. 将旧数据拷贝到新空间，释放旧空间

整型动态数组

```
#include <stdio.h>
#include <stdlib.h>

int *insert(int *count, int *arr, int num)
{
    int *new = (int *)malloc(sizeof(int) * (*count + 1));
    if (new == NULL)
        return NULL;

    // 拷贝旧数据
    memcpy(new, arr, sizeof(int) * (*count));

    // 新数据放到末尾
    new[*count] = num;
    (*count)++;

    free(arr);
    return new;
}

int main(void)
{
    int count = 0;
    int *arr = NULL;

    for (int i = 0; i < 10; i++)
    {
        int num = rand() % 100;
        arr = insert(&count, arr, num);
    }

    for (int i = 0; i < count; i++)
        printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    return 0;
}
```

结构体动态数组

```
struct cls_t {
    char name[64];
    int id;
    int age;
    char sex;
    float score;
};

struct cls_t *insert(int *count, struct cls_t *prev, struct cls_t cls)
{
    // 申请新空间 (比原来多1个)
    struct cls_t *new = (struct cls_t *)malloc(
        sizeof(struct cls_t) * (*count + 1));

    // 拷贝旧数据
    memcpy(new, prev, sizeof(struct cls_t) * (*count));

    // 追加新数据
    new[*count] = cls;
    (*count)++;

    free(prev);
    return new;
}
```

泛型动态数组 (void*封装)

使用 void* 实现通用的动态数组，可以适配任意类型：

```

// 通用插入函数
void *insert(int *count, void *prev, void *data, int size)
{
    // 申请新空间
    void *new = malloc(size * (*count + 1));
    if (new == NULL) return NULL;

    // 拷贝旧数据
    memcpy(new, prev, size * (*count));

    // 追加新数据 (注意偏移计算)
    memcpy(new + *count * size, data, size);
    (*count)++;

    free(prev);
    return new;
}

// 使用int类型
int count = 0;
int *arr = NULL;
int num = 123;
arr = insert(&count, arr, &num, sizeof(int));

// 使用结构体类型
struct cls_t cls;
struct cls_t *students = NULL;
students = insert(&count, students, &cls, sizeof(struct cls_t));

```

更新日志：

- 2026-05-24：初始版本，包含数据类型、运算符、流程控制等内容
- 2026-05-29：新增break/continue、死循环、延迟函数、随机数、goto语句
- 2026-05-30：新增函数（定义、传参、不定参、递归）、分文件编程、编译过程
- 2026-06-07：新增Makefile详解（变量、依赖、自动变量、.PHONY）、动态库和静态库创建与使用
- 2026-06-17：新增字符串函数（strcpy/strcat/strcmp/strstr等）、内存函数（memcpy/memmove/memset）
- 2026-06-18：新增二维数组、数组sizeof计算、地址偏移、fgets、头文件防重复包含
- 2026-07-04：新增VT码与终端控制、read键盘输入、方向键处理

- 2026-07-05: 新增指针、内存布局（栈/堆/数据段）、malloc/calloc/realloc/free、修饰符（static/const/volatile/extern/register）
- 2026-07-11: 新增多级指针、数组指针、指针数组
- 2026-07-12: 新增网络配置、down_up.sh自动下载上传
- 2026-07-13: 新增指针函数、函数指针、函数指针数组、结构体（定义/访问/数组/指针/内存/柔性数组/对齐/位域/参数/嵌套）、共用体、枚举
- 2026-07-14: 新增预处理（宏定义/#运算符/##运算符/宏vs函数/有返回值宏/容错宏/条件编译/特殊字符）、文件IO（fopen/fclose/fgetc/fputc/fgets/fputs/cp实现）
- 2026-07-15: 新增fread/fwrite、fprintf/fscanf、fseek/ftell/rewind、grep实现、动态数组（int/结构体/void*泛型）
- 2026-07-16: 新增容错处理函数（perror/errno/strerror）

29. 容错处理函数

perror — 打印错误信息

```
#include <stdio.h>
```

```
void perror(const char *s);
```

- 自动根据全局变量 `errno` 的值，打印对应的错误描述
- 参数 `s` 是自定义前缀，输出格式：`s: 系统错误信息`
- 需要包含 `<errno.h>` 获取错误号

errno — 全局错误号

```
#include <errno.h>
```

```
// errno 是全局变量，记录最后一次错误的编号
```

```
// 每个错误号对应一种错误类型
```

strerror — 错误号转字符串

```
#include <string.h>
```

```
char *strerror(int errnum);
```

- 将错误编号转换为可读的错误描述字符串
- 返回的字符串不需要手动释放

文件操作中的错误处理示例

```
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main(int argc, char *argv[])
{
    FILE *fp = NULL;

    fp = fopen(argv[1], "r");
    if (fp == NULL)
    {
        perror("fopen");                // 输出: fopen: No such file or director
        printf("errno : %d\n", errno);    // 输出错误号
        printf("strerror : %s\n", strerror(errno)); // 同上
        goto ERR1;
    }

    fclose(fp);
    return 0;
ERR1:
    return -1;
}
```

三种错误处理方式对比

方式	说明
perror(s)	直接打印前缀 + 系统错误信息，简单快捷
errno	全局变量，记录错误编号，配合 strerror 使用
strerror(errno)	将错误号转为字符串，灵活控制输出格式

30. 练习题

练习1：闰年判断

```
#include <stdio.h>

int main(void)
{
    int year;

    printf("请输入年份: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d年是闰年, 共366天\n", year);
    else
        printf("%d年不是闰年, 共365天\n", year);

    return 0;
}
```

练习2：求最大值

```
#include <stdio.h>

int main(void)
{
    int a, b;

    printf("请输入两个整数: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("最大值是: %d\n", a);
    else
        printf("最大值是: %d\n", b);

    return 0;
}
```

练习3：求绝对值

```
#include <stdio.h>

int main(void)
{
    int num;

    printf("请输入一个整数: ");
    scanf("%d", &num);

    if (num >= 0)
        printf("绝对值是: %d\n", num);
    else
        printf("绝对值是: %d\n", -num);

    return 0;
}
```

练习4：成绩等级判断

```
#include <stdio.h>

int main(void)
{
    int score;

    printf("请输入成绩: ");
    scanf("%d", &score);

    if (score >= 90)
        printf("等级: A\n");
    else if (score >= 75)
        printf("等级: B\n");
    else if (score >= 60)
        printf("等级: C\n");
    else
        printf("等级: D\n");

    return 0;
}
```

练习5：大小写转换

```
#include <stdio.h>

int main(void)
{
    char ch;

    printf("请输入一个字符: ");
    scanf("%c", &ch);

    if (ch >= 'A' && ch <= 'Z')
        printf("转换后: %c\n", ch + 32);    // 大写转小写
    else if (ch >= 'a' && ch <= 'z')
        printf("转换后: %c\n", ch - 32);    // 小写转大写
    else
        printf("原样输出: %c\n", ch);

    return 0;
}
```


练习6：象限判断

```
#include <stdio.h>

int main(void)
{
    float x, y;

    printf("请输入坐标(x, y): ");
    scanf("%f %f", &x, &y);

    if (x > 0 && y > 0)
        printf("第一象限\n");
    else if (x < 0 && y > 0)
        printf("第二象限\n");
    else if (x < 0 && y < 0)
        printf("第三象限\n");
    else if (x > 0 && y < 0)
        printf("第四象限\n");
    else
        printf("在坐标轴上\n");

    return 0;
}
```

练习7：商场促销计算

```
#include <stdio.h>

int main(void)
{
    int count;
    float price = 1999;
    float discount;

    printf("请输入购买台数: ");
    scanf("%d", &count);

    if (count >= 20)
        discount = 0.7;
    else if (count >= 15)
        discount = 0.8;
    else if (count >= 10)
        discount = 0.85;
    else if (count >= 5)
        discount = 0.9;
    else if (count >= 3)
        discount = 0.95;
    else
        discount = 1.0;

    float total = count * price * discount;

    printf("总价: %.2f元\n", total);

    return 0;
}
```